Temporal Segmentation Changes

23 October 2026 | TYM v0.9 release report

Status: published on Azure and included in the consolidated GitHub release. The API and both UI hosts use v0.9 temporal-development segmentation. This report describes code changes and regression evidence, not measured NLP accuracy. Production database caching remains disabled pending persistent storage.

From Phrases to Temporal Developments

Sentence and clause detection now supplies lower-level event candidates. It does not define time segments. A TS groups consecutive events within a retained temporal/development context and can span many sentences and paragraphs.

Before	After
Actor-list, location and paragraph features could create boundaries.	Those changes alone cannot create a TS. A shared scene or journey may remain one TS.
Adjacent-event comparison could lose earlier time context.	Supported anchors persist through unanchored events; calendar precision is retained.
Learned boundaries could follow structural features.	All algorithms require temporal/development evidence. Explicit context shifts force a boundary; unsupported splits are vetoed.
The inspector showed scores and features.	It also shows temporal frame, retained time reference and the basis for the final decision.

Basis in the Supplied Papers

Cristea's **The Time Yards Model: Rethinking the Way to Look at Texts** (SYNASC 2016), Section II, pages 1-3, defines a TS as a contiguous development with unity of actors, time, space, type and perspective. A journey can involve a chain of locations; communication can couple actors in different places. Events are not automatically separate TSs. The illustration supplied with the paper distinguishes text order from story-time tracks.

The implementation uses these distinctions as a design basis, not as a claim that the paper supplies this algorithm. Section III, page 5, describes manual annotation and a future automatic parser. Recognizing full actor coupling and inferred perspectives still requires further work here. The older Cristea and Macovei paper, **Why Time Resembles Rail Yards? A Way to Look at Time in Text Books**, Sections 2-3, supplies the earlier diagram/type conventions.

The existing NAR/REM/SUP/GEN/FIC output taxonomy remains compatible. It has not been replaced by SYNASC's simplified NAR/SUP/GEN taxonomy, where narrated flashbacks can be NAR. Temporal frame and TS type are distinct output concepts.

Implemented Boundary Policy

- Retain supported calendar, story-day, weekday and clock references across events. Validate ISO dates with DateOnly; treat a date inside a known year as compatible. Do not let a later year-only mention erase a finer known date.
- Detect bounded English/Romanian cues for flashback entry, narrative-time return, explicit forward jumps and parallel developments. Ordinary then/apoi progression, durations and changes in entity lists alone are not boundaries.
- Recognize explicit changes of perspective, hypothetical/general world plane, communication or separation. This is a small cue inventory, not full discourse understanding or robust automatic scene recognition.
- Carry standalone temporal headings to the following recognized event. TS text is an exact original UTF-16 slice, preserving punctuation and paragraphs.
- Keep rules, ML.NET logistic regression and averaged perceptron selectable. Both learned models use 48 synthetic temporal-feature patterns. Explicit-shift and continuity guards constrain their decisions; weaker tense candidates are classified by the selected method. Scores are raw, uncalibrated model outputs, not probabilities of the final guarded boundary.

Splitting at an explicitly signalled forward jump is an implementation policy, not a requirement of the paper: linear developments can contain temporal gaps. Relative references remain symbolic; no absolute date is invented from earlier, later or now without a known calendar reference.

Worked Example

English input: Adam waited at home. Years earlier, Adam had traveled to Paris. Margaret had waited there. Now, Adam opened the door. Adam searched the room.

TS	Events	Temporal interpretation
TS1	Adam waited at home.	Initial narrative development.
TS2	Years earlier, Adam had traveled to Paris. Margaret had waited there.	One retrospective development across two sentences.
TS3	Now, Adam opened the door. Adam searched the room.	Return to narrative time, continuing across two sentences.

All three algorithms produce **five events and three TSs** for this authored example. A Romanian equivalent using Cu ani in urma and Acum also produces three TSs in the regression suite. A separate 180-event scene spread across 60 paragraphs remains **one TS** under all three algorithms. These are reproducible fixture outputs, not a literary segmentation-quality benchmark.

API, Viewer and Cache Changes

The compatibility option IDs remain article_rules, mlnet_logistic and mlnet_perceptron. Responses now declare `segmentation.unit = temporal_development`. Each decision includes `basis`, `frame` and `time_reference`, alongside event/TS IDs, offsets, observed features and an optional raw model score.

The English/Romanian source inspector exposes those fields. The default menu label is Temporal continuity rules. The v0.9 UI rejects APIs that do not advertise temporal-development semantics. The API model namespace and browser data/shell versions are bumped so earlier phrase-like results are not silently reused. Changing TT density preserves TS decisions and source spans; it is not a new segmentation run. Existing source-access and export restrictions remain.

Verification

- Release API/checker and UI builds pass without warnings or errors.
- 93 authored temporal-development algorithm cases cover coherent scenes, journeys, flashbacks/returns, forward shifts, anchors across unanchored events, calendar precision, invalid dates, headings, perspectives, interaction changes, durations, exact source spans and density invariance.
- Six bilingual algorithm fixtures check score semantics, event coverage, UTF-16 boundaries, repeated/concurrent predictions and density-cache consistency.
- Existing checks cover trained English/Romanian NER, three POS fixtures, nineteen reference fixtures and 202 density values. All 51 Node tests pass.
- Browser regressions cover 36 language/algorithm/theme/viewport cases, source paging, chapter offsets, old-semantics rejection and temporal-context fields. Bilingual selector/header alignment and offline reload/cache checks remain.
- SQLite tests verify persistence/expiry/eviction and an isolated HTTP restart with nine progressive views and fresh session IDs. No private book is used as a gold benchmark, and no validation email or paid model service is involved.

Remaining Limits and Next Work

This is a conservative non-LLM prototype, not the complete paper model. Upstream rule-based event extraction can miss predicates. Unmarked scene changes, indirect perspectives, quotation scope, nested memories, coreference and mixed time expressions within one clause are not reliably resolved. Clock/day expressions have no full calendar or narrative-reference resolver.

TT identity, joins/splits, chronological placement and TimeML relations retain their existing heuristic machinery. The rail-yard illustration is a semantic target, not a claim that full book chronology is now reconstructed correctly. Build an adjudicated English/Romanian TS corpus, preserve uncertain relations, separate canonical story-time constraints from display grouping, and evaluate boundary F1, WindowDiff/Pk and relation quality on held-out complete works.

Code: TemporalDevelopmentContext.cs, Segmentation.cs, Program.cs, ui/Tym.Ui/wwwroot/app.js and tools/Tym.GenerationChecks/TemporalSegmentationChecks.cs. See docs/segmentation-classifiers.md for API details and reproduction commands.